

BLUESTOCK MUTUAL FUND ANALYTICS CAPSTONE PROJECT

Submitted by: Mugilan M

Duration: 7-Day Mutual Fund Analytics Project

Technologies: Python • Pandas • SQLite • SQL • Power BI • GitHub

Submission Date: June 2026

Executive Summary

For this capstone, I built a full analytics pipeline around mutual fund performance data — from raw CSV ingestion all the way to an interactive Power BI dashboard. The goal was straightforward: give investors a single place to evaluate fund performance, compare risk levels, and spot meaningful trends without having to juggle spreadsheets or dig through data manually.

I used Python and Pandas to clean and process the data, then loaded everything into an SQLite database so I could query it efficiently with SQL. From there, I calculated key financial metrics like Alpha, Beta, Sharpe Ratio, and Risk Grade to go beyond surface-level return numbers. The final Power BI dashboard ties it all together, letting users filter by category, compare funds side by side, and assess risk at a glance.

What this project gave me, beyond the technical output, was a working understanding of how financial data flows from raw source to actionable insight — which is exactly the kind of end-to-end thinking this capstone was designed to build.

Problem Statement

Anyone who’s tried to pick a mutual fund knows how overwhelming it gets. There’s a huge amount of data across funds, and the metrics used to evaluate them — returns, alpha, beta, expense ratios — aren’t always easy to interpret, especially when you’re comparing dozens of options at once.

The gap I wanted to address: investors often rely too heavily on raw return numbers, which don’t tell the whole story. A fund with high returns might carry excessive risk or consistently underperform its benchmark on a risk-adjusted basis. I wanted to build something that surfaces those nuances in a clear, visual way — a system that makes multi-factor fund evaluation accessible without requiring deep financial expertise.

Objectives

- Collect, clean, and organize mutual fund data for analysis
- Store and manage the cleaned dataset in an SQLite database
- Run exploratory analysis to understand fund characteristics and distributions
- Calculate performance metrics including Alpha, Beta, and Sharpe Ratio
- Build a risk classification system to grade funds by risk profile
- Develop an interactive Power BI dashboard for end-user exploration
- Extract business-relevant insights from the data
- Demonstrate a complete, reproducible end-to-end analytics workflow

Tools and Technologies

Technology	Purpose
Python	Core data processing and analytics scripting
Pandas	Data manipulation, cleaning, and transformation

Technology	Purpose
SQLite	Lightweight relational database for structured storage
SQL	Querying, filtering, and data retrieval
Matplotlib	Exploratory charts and inline visualizations
Power BI	Interactive dashboard for business reporting
GitHub	Version control and project hosting
Jupyter Notebook	Development and documentation environment

Data Collection and Understanding

The dataset I worked with covers mutual fund performance, categories, return histories, and risk attributes. Before doing anything analytical, I spent time genuinely understanding what I was working with — reviewing column names, checking data types, looking for missing values, and spotting inconsistencies that could distort later calculations.

This upfront inspection wasn't just housekeeping. Understanding the variables early shaped every downstream decision: which metrics were worth calculating, how to handle gaps in the data, and what kinds of comparisons would actually be meaningful in the dashboard.

```
fund_master.head()
```

[4]

... Shape: (40, 15)

...

	amfi_code	fund_house	scheme_name	category	sub_category	plan	launch_date	benchmark	expense_ratio_pct	exit_load_pct	min_sip_amount
0	119551	SBI Mutual Fund	SBI Bluechip Fund - Regular Plan - Growth	Equity	Large Cap	Regular	2006-02-14	NIFTY 100 TRI	1.54	1.0	500
1	119552	SBI Mutual Fund	SBI Bluechip Fund - Direct Plan - Growth	Equity	Large Cap	Direct	2013-01-01	NIFTY 100 TRI	0.66	1.0	500
2	119598	SBI Mutual Fund	SBI Small Cap Fund - Regular Plan - Growth	Equity	Small Cap	Regular	2009-09-09	BSE 250 SmallCap TRI	1.43	1.0	500
3	119599	SBI Mutual Fund	SBI Small Cap Fund - Direct Plan - Growth	Equity	Small Cap	Direct	2013-01-01	BSE 250 SmallCap TRI	0.72	1.0	500
4	119120	SBI Mutual Fund	SBI Magnum Gilt Fund - Regular Plan - Growth	Debt	Gilt	Regular	2000-12-30	CRISIL Dynamic Gilt Index	0.77	0.0	500

min_lumpsum_amount	fund_manager	risk_category	sebi_category_code
1000	Sohini Andani	Moderate	EC01
1000	Sohini Andani	Moderate	EC01
1000	R. Srinivasan	Very High	EC03
1000	R. Srinivasan	Very High	EC03
1000	Dinesh Ahuja	Low	DC02

```

fund_master.info()

[5]

<class 'pandas.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   amfi_code             40 non-null    int64  
 1   fund_house            40 non-null    str     
 2   scheme_name           40 non-null    str     
 3   category              40 non-null    str     
 4   sub_category          40 non-null    str     
 5   plan                  40 non-null    str     
 6   launch_date           40 non-null    str     
 7   benchmark             40 non-null    str     
 8   expense_ratio_pct     40 non-null    float64 
 9   exit_load_pct         40 non-null    float64 
10   min_sip_amount        40 non-null    int64  
11   min_lumpsum_amount    40 non-null    int64  
12   fund_manager          40 non-null    str     
13   risk_category         40 non-null    str     
14   sebi_category_code    40 non-null    str     
dtypes: float64(2), int64(3), str(10)
memory usage: 4.8 KB

```

```
fund_master.isnull().sum()

[6]

...    amfi_code      0
      fund_house      0
      scheme_name      0
      category        0
      sub_category     0
      plan            0
      launch_date      0
      benchmark        0
      expense_ratio_pct 0
      exit_load_pct    0
      min_sip_amount    0
      min_lumpsum_amount 0
      fund_manager      0
      risk_category     0
      sebi_category_code 0
      dtype: int64
```

ETL Process

Extract

Raw mutual fund data was brought into the Python environment using Pandas. This served as the single source of truth for all subsequent processing. I kept this step simple and repeatable — no manual file edits, just a clean import that could be re-run if the source data changed.

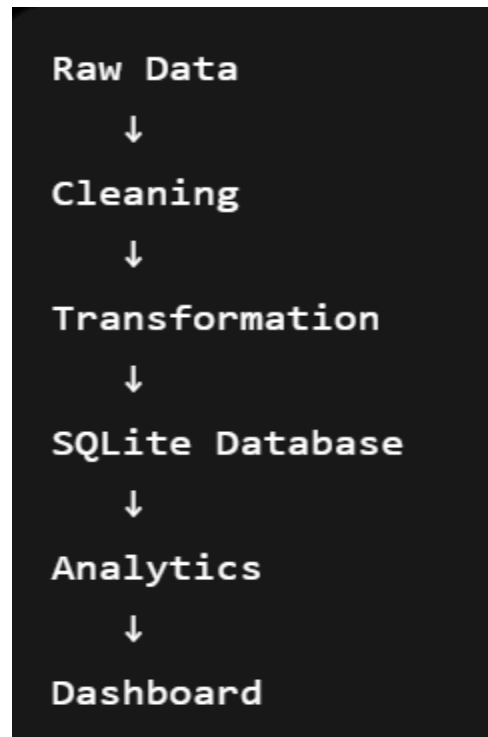
Transform

The bulk of the work happened here. I addressed missing values, standardized column formats to ensure consistency, validated numerical fields against expected ranges, and computed derived metrics needed for performance analysis. A few fields required careful judgement calls — for example, deciding how to handle funds with incomplete return histories without skewing aggregate comparisons.

Load

Once the data was clean, I loaded it into an SQLite database. This made it much easier to query specific segments — filtering by category, pulling top performers by Alpha, or slicing by risk grade — without reloading the full dataset in Python each time. The ETL pipeline as a whole gave me a reliable, consistent foundation that the rest of the project depended on.

Workflow



```
df = pd.read_csv(RAW_PATH / "01_fund_master.csv")
print("Original Shape:", df.shape)

# Remove duplicates
df = df.drop_duplicates()

# AMFI code as string
df["amfi_code"] = df["amfi_code"].astype(str)

# Convert date
df["launch_date"] = pd.to_datetime(
    df["launch_date"],
    errors="coerce"
)

# Clean text columns
text_cols = [
    "fund_house",
    "scheme_name",
    "category",
    "sub_category",
    "plan",
    "benchmark",
    "fund_manager",
    "risk_category",
    "sebi_category_code"
]

for col in text_cols:
    df[col] = df[col].astype(str).str.strip()

# Save cleaned file
df.to_csv(
    PROCESSED_PATH / "01_fund_master_cleaned.csv",
    index=False
)

print("Saved Successfully")
print(df.shape)
```

[5]

```
... Original Shape: (40, 15)
Saved Successfully
(40, 15)
```

```
import sqlite3

conn = sqlite3.connect("../bluestock_mf.db")

print("Database Created Successfully!")

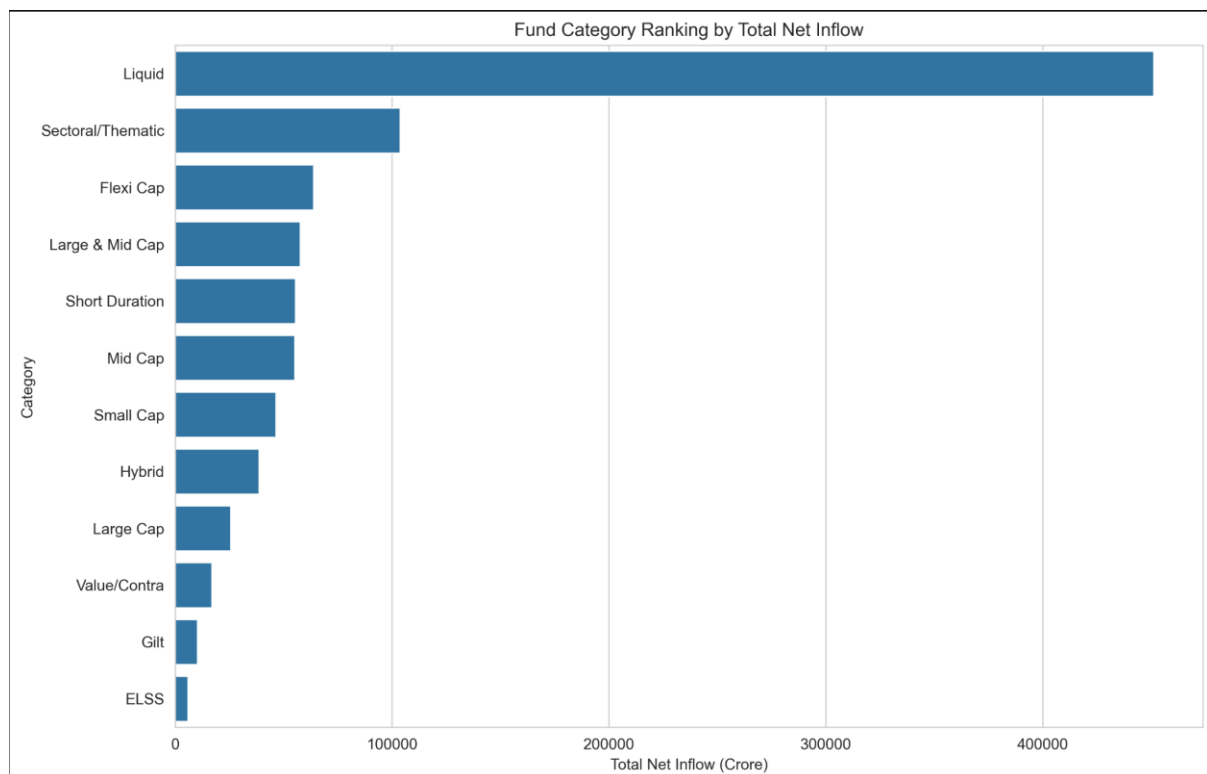
... Database Created Successfully!

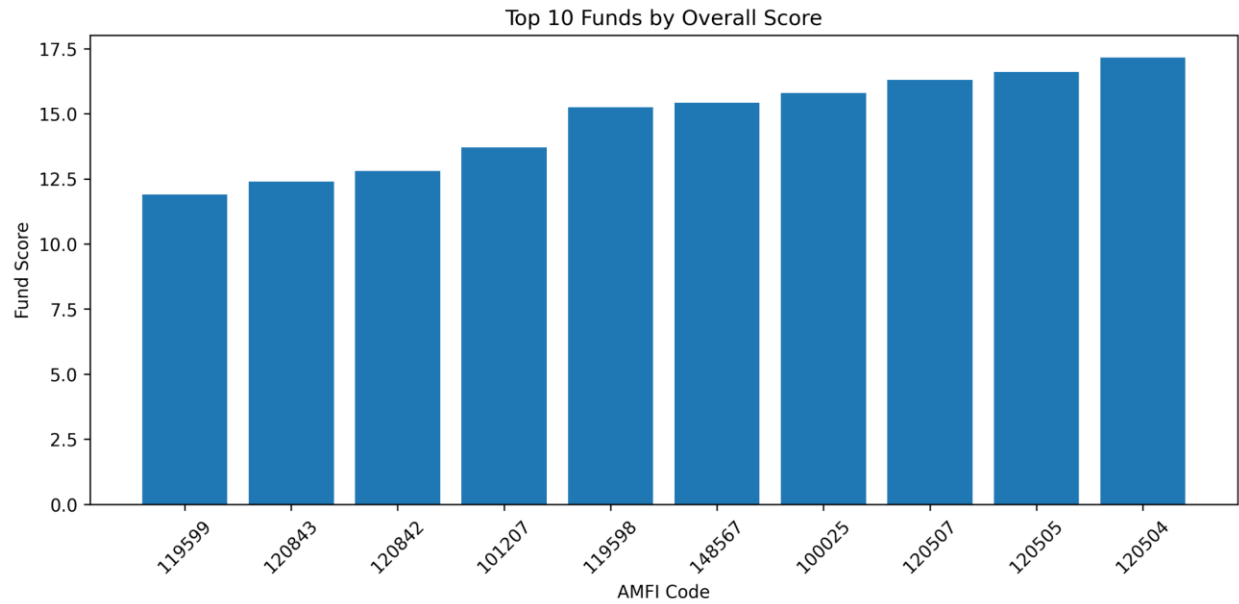
import pandas as pd
import sqlite3

conn = sqlite3.connect("../bluestock_mf.db")

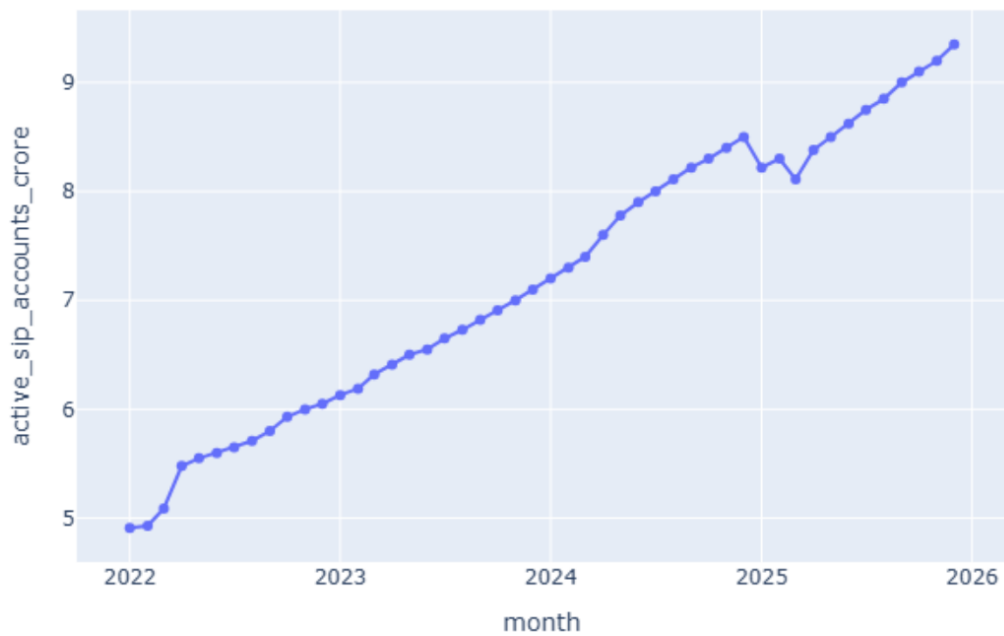
print("Connected to Database")

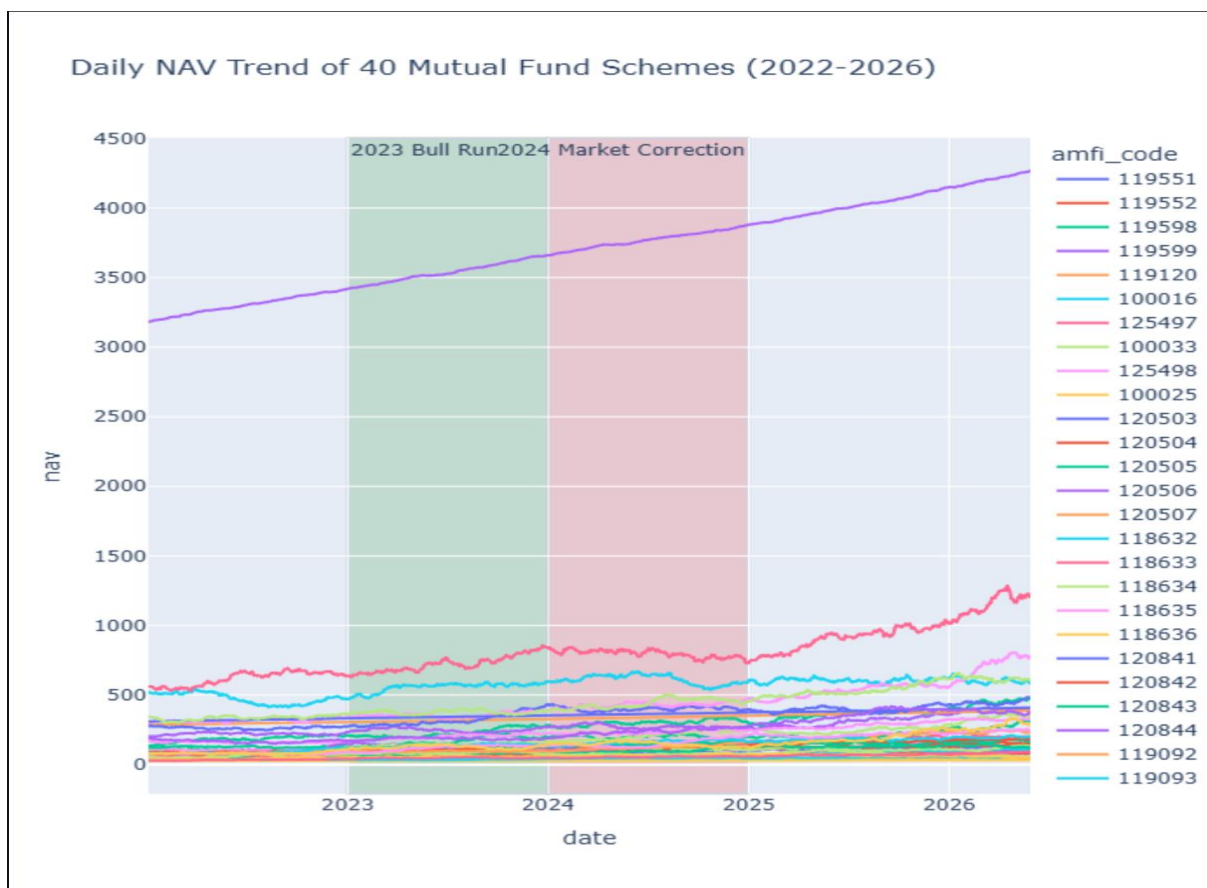
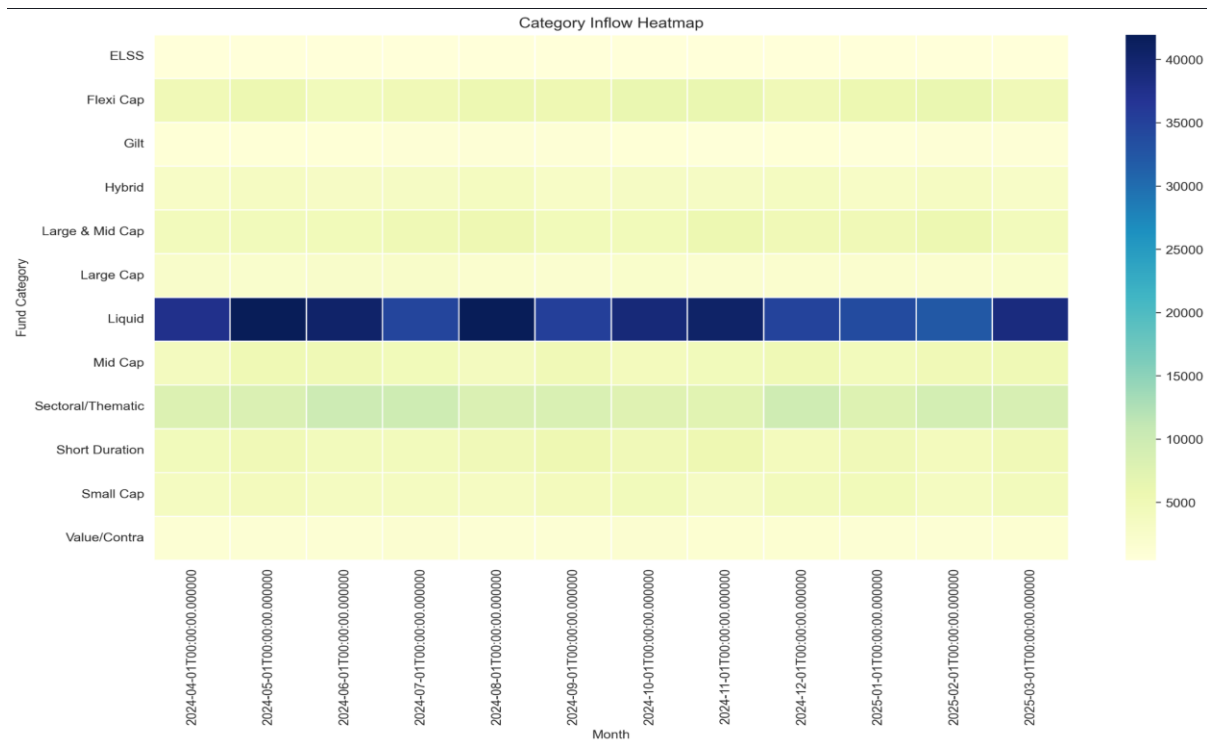
... Connected to Database
```

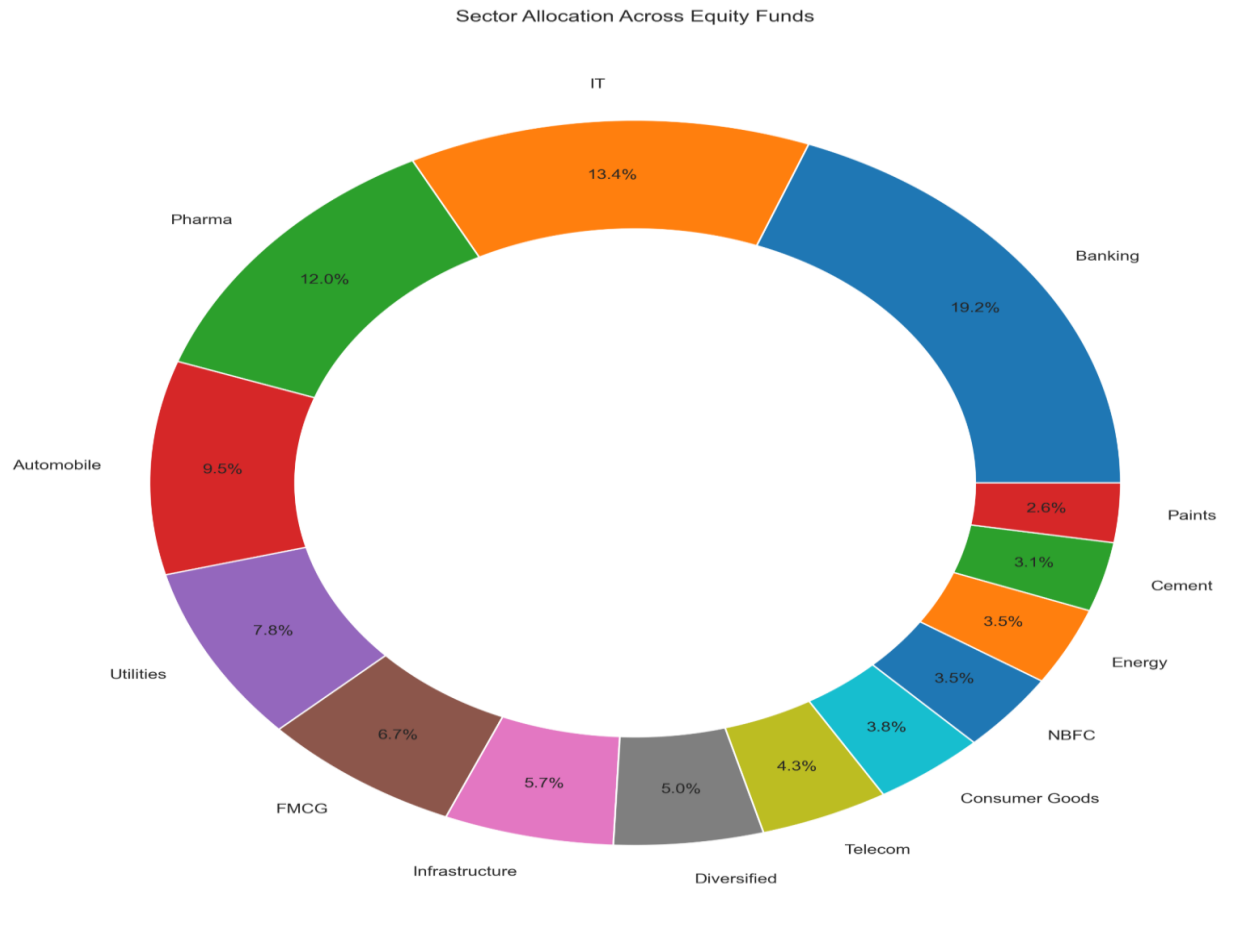
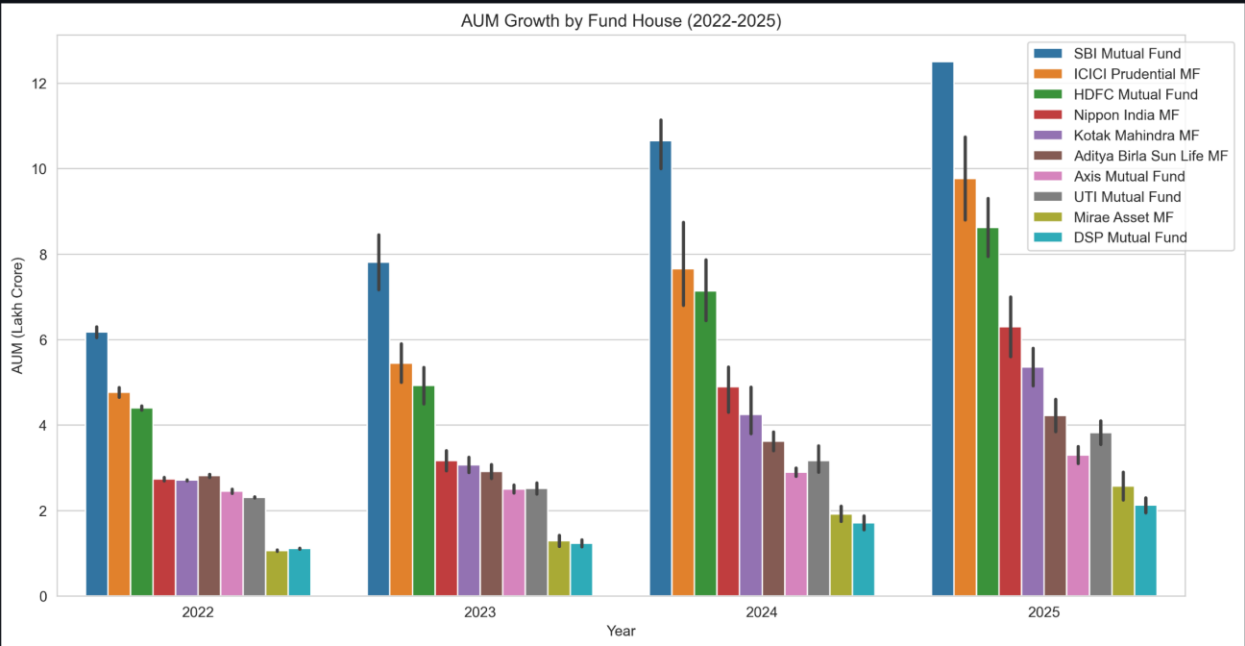




Growth in Active SIP Accounts







Exploratory Data Analysis

With clean data in hand, I ran an EDA to get a real feel for the dataset before diving into performance metrics. I generated distributions, summary statistics, and category breakdowns to understand where the data was concentrated and where there were interesting outliers worth investigating.

A few things stood out early: category representation was uneven, with some fund types appearing far more often than others. Return distributions showed significant spread, which reinforced the need for risk-adjusted metrics rather than raw return rankings. These observations directly influenced which visualizations I prioritized in the dashboard.

More than anything, EDA was about asking questions of the data before assuming I knew what it contained. That mindset — looking before concluding — kept the analysis honest throughout.

Performance Analysis

Return numbers alone don't capture how well a fund is actually performing. A fund that returns 15% in a year where its benchmark returned 18% has underperformed, even though the absolute number looks good. That's the problem performance metrics are designed to solve.

Alpha

Alpha measures how much a fund's return deviates from what its benchmark would predict. I calculated Alpha for each fund to identify which ones were genuinely adding value through active management versus simply riding market momentum. Positive Alpha funds became a key segment in the dashboard.

Beta

Beta captures how sensitive a fund is to overall market movements. A high-Beta fund amplifies market swings in both directions; a low-Beta fund moves more independently. I used Beta to help classify funds by their risk behavior and inform the risk grading system.

Risk Grade

Rather than leaving risk as a raw number, I introduced a grading system that buckets funds into interpretable categories. This makes it practical for a non-technical user to quickly understand a fund's risk profile without needing to interpret Beta directly.

Fund Scorecard

To make comparison easier, I built a consolidated scorecard that combines Alpha, Beta, returns, and risk grade into a single summary view per fund. Think of it as a one-row snapshot of how a fund is performing across multiple dimensions simultaneously.

Dashboard Analysis

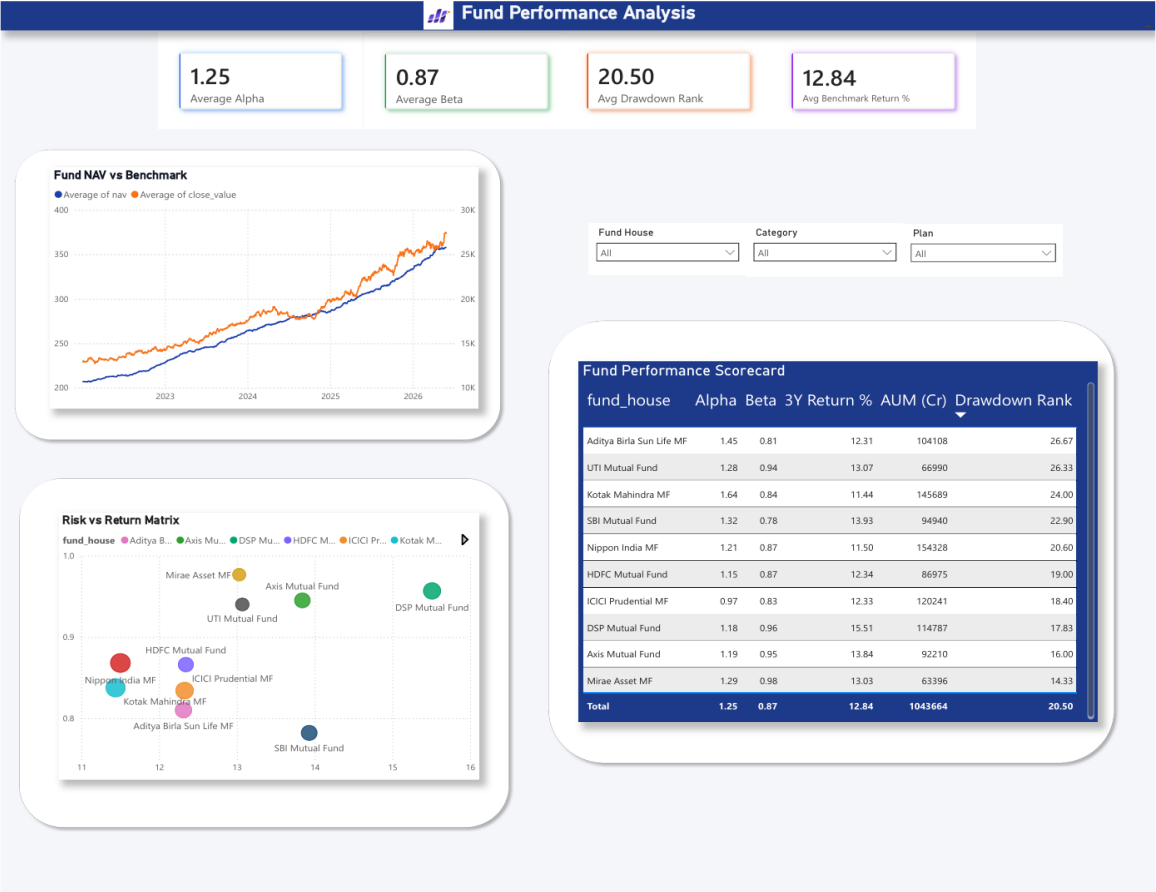
The Power BI dashboard is where all the analysis becomes usable. I designed it around the question: "what does an investor actually need to see to make a better decision?" That led me to prioritize comparison, filtering, and risk visibility over visual complexity.

The dashboard includes category-wise fund distribution, performance comparisons across Alpha and returns, risk classification breakdowns, and benchmark-relative visualizations.

Interactive slicers let users zero in on specific fund types, risk grades, or performance tiers without wading through the full dataset.

A design decision I'm particularly happy with: I kept the default view high-level, with drill-down available for users who want more detail. That hierarchy means the dashboard works for both quick scans and deeper analysis sessions.





Key Findings

- Fund category distribution is skewed — a small number of categories dominate the dataset, which matters when generalizing trends
- Performance varies widely across funds, confirming that return alone is an insufficient evaluation criterion
- Risk profiles differ substantially even within the same category, reinforcing the need for individual fund-level risk assessment
- Some funds consistently generate returns above benchmark expectations, which shows up clearly in positive Alpha values
- Risk-adjusted metrics provided a meaningfully different ranking than raw return performance in several cases
- Combining analytics and interactive visualization made it significantly easier to spot patterns that wouldn't have surfaced from static tables

Recommendations

- Evaluate funds using both absolute returns and risk-adjusted metrics — the difference often reveals which funds are actually worth their risk
- Benchmark comparison should be a standard part of fund review, not an afterthought
- Diversifying across fund categories reduces concentration risk and can smooth out portfolio volatility
- Match investment selections to risk tolerance explicitly — the risk grading system built in this project makes this more systematic
- Use interactive dashboards for ongoing monitoring, not just one-time analysis; market conditions shift and so do fund rankings
- Future iterations could incorporate real-time data feeds and additional financial indicators to make the system more dynamic

Limitations

This project has a few honest limitations worth acknowledging. First, the analysis is only as good as the source data. Any gaps or inaccuracies in the dataset flow downstream into the metrics and dashboard.

Second, the dataset represents a snapshot in time. Mutual fund performance is shaped by market cycles, macroeconomic conditions, and events that a historical dataset may not fully capture. Some funds may look strong or weak in ways that don't hold outside the period represented.

Third, some metrics were computed using simplified assumptions. Real-world performance analysis often involves more nuanced models and access to more granular data (daily NAV, expense ratios, tax treatment, etc.) that weren't available here.

Finally, this dashboard is an analytical tool, not financial advice. The findings should inform investment thinking, not replace it.

Conclusion

This capstone gave me the chance to build something I actually believe is useful — a system that makes mutual fund evaluation more accessible and more rigorous at the same time. By connecting data engineering, SQL, financial metrics, and business intelligence into a single pipeline, I got hands-on experience with the kind of end-to-end analytical workflow that shows up in real data analytics roles.

The Power BI dashboard isn't just a deliverable — it's a working example of how complex financial data can be transformed into something an investor can actually use. The performance metrics I computed (Alpha, Beta, risk grades) add analytical depth that raw return numbers can't provide on their own.

Looking back, the most valuable part of this project wasn't any single technical step — it was learning to make thoughtful decisions at each stage of the pipeline, from how to handle messy data to how to design a dashboard for a non-technical audience. Those judgement calls are what separate analysis that works in theory from analysis that works in practice.